ARCHITECTURE RECOVERY

Part 1: What is Software Architecture?

Simple Definition:

Software architecture is the basic structure of a system. It includes the parts of the system, how they connect to each other, and the rules for building and changing it.

What does architecture include?

1. The parts (components) – like modules, databases, user interfaces
2. How parts connect (relationships) – like one part calling another part
3. Rules (constraints) – like "this part cannot talk directly to that part"

In very simple words:

Architecture is the set of important decisions that stop developers from doing whatever they want. It keeps the system organized.

Part 2: Architectural Viewpoints

Simple Definition:

A viewpoint is like a camera lens or a template. It helps you look at the system from one specific angle for one specific audience.

Key Idea:

- Viewpoint = The template (reusable)
- View = The actual picture you draw (one time use)

Why different viewpoints?
Because different people care about different things.

Viewpoint	What it shows	Who cares?

Run-time	Which part does which job while the program runs	Developers
Process	How many threads run at the same time	Performance experts
Dataflow	How data moves from one place to another	Data experts
Deployment	Which part runs on which computer	IT / Operations team
Module	How code is organized into folders and files	Programmers
Build	Which parts depend on which other parts	Build engineers
File	How files are stored on disk	Anyone

Simple example:

- A teacher wants the "Module viewpoint" (how code is organized)
- A network admin wants the "Deployment viewpoint" (which server runs what)

Both use different templates (viewpoints) to get what they need.

Part 3: Architectural Styles

Simple Definition:

An architectural style is a common pattern or a family design that many systems follow.

What does a style include?

1. Components – What are the parts?
2. Connectors – How do parts talk to each other?
3. Constraints – What are the rules for putting parts together?
4. Semantic models – Why does this pattern work?

Part 4: Some Classical Styles (Common Patterns)

1. Layered Style

- What it is: The system is divided into layers. Each layer only talks to the layer directly below it.
- Example: A cake has layers. You eat from top to bottom.
- Rule: Upper layers can call lower layers. Lower layers cannot call upper layers directly (they use callbacks).

2. Client-Server Style

- What it is: One part is the server (provides service). Another part is the client (asks for service).
- Roles are unbalanced: Server is boss. Client is worker.
- Types:
    - Fat client = Client does a lot of work
    - Thin client = Client does very little work

3. Tiered Style

- What it is: The system is split into tiers (peers). Each tier has its own job.
- Common three tiers:
    - Presentation tier – What the user sees (screen, buttons)
    - Logic tier – The business rules (calculations, decisions)
    - Data tier – Where data is stored (database)

4. Dataflow Style

- What it is: Data moves only in one direction (downstream).
- Examples:
    - Pipes and filters (like a water pipe)
    - Batch sequential (do step 1, then step 2, then step 3)

5. Blackboard Style

- What it is: All tools share one common place (the blackboard) to coordinate.
- Example: Several experts gather around a whiteboard. Each writes their idea. Everyone reads it.

Part 5: Architecture Recovery (The Main Topic)

Simple Definition:

Architecture recovery is the process of figuring out the architecture of an existing system by looking at its code, how it runs, and any old documents.

Why do we need it?

- Because old documents are often wrong or outdated
- The actual code may be very different from the original plan
- To understand a legacy system before fixing or upgrading it

The 4 Steps of Architecture Recovery:

Step	What you do	Simple words

1. Extraction	Read the source code, config files, build scripts	Gather all the raw information
2. Storage	Put all facts into a database	Store everything you found
3. Abstraction	Group small pieces into bigger patterns	Find the big picture from small details
4. Presentation	Draw diagrams (views) for different audiences	Show what you found

---

Part 6: How to Recover Architecture (Techniques)

Technique 1: Static Analysis

- What it is: Looking at the code without running it
- What you check:
  - Package names (like `com.app.ui`, `com.app.database`)
  - Import statements
  - Which class inherits from which
- Limit: Cannot see runtime behavior (like plugins)

Technique 2: Dynamic Analysis

- What it is: Running the program and watching what happens
- How: Using debuggers or profilers

- Good: Shows real interactions
- Bad: Only shows what happened during that specific run

Technique 3: Clustering

- What it is: Using math/algorithms to group related code into subsystems
- Idea: If two functions talk to each other a lot, they belong together

---

Part 7: Common Problems (What Can Go Wrong)

Problem	What it means
Erosion	The real code is different from the documentation
Missing views	You recover one view (modules) but not another (processes)
Noise	Too many small details hide the big picture
Big ball of mud	The system has no style at all. It is just a mess.

Part 8: Example (Putting It All Together)

Scenario: You have an old Java project with no documentation.

1. Extract: You scan the code. You see 50 classes using `DatabaseConnection`
2. Abstract: A tool groups these 50 classes into a "Data Layer"
3. Check viewpoints:
    - Deployment view → All run on one Tomcat server
    - Module view → UI calls Logic calls Data (no skipping)
4. Identify style: This is a 3-Layered architecture
5. Conclusion: The original document said "Peer-to-Peer" but you recovered "Layered Client-Server"

-